

KGP Gyankosh: Complete Technical Documentation

Comprehensive File Directory, System Architecture, & Line-by-Line Code Breakdown

Institution: Kashmir Government Polytechnic College, Srinagar

Capstone Project: IIT Patna — Advanced Enterprise RAG & Generative AI Platform

Scope of Document: Exhaustive documentation of all codebase files, functional operations, internal logic, algorithms, and line-by-line code walk-through.

1. Executive Overview & Two-Stage System Architecture

KGP Gyankosh is an internal administrative intelligence assistant developed for Kashmir Government Polytechnic College, Srinagar. The system completely decouples the heavy offline indexing pipeline from the online interactive search engine:

- **Stage 1: Offline Batch Ingestion & Dual Indexing Pipeline:** Executed via `build_index.py`. Recursively crawls `data/`, calculates MD5 hashes for incremental change tracking, routes scanned PDFs through RapidOCR, splits documents into 500-character semantic chunks with 100-character overlap, and synchronizes a dense FAISS vector index (MiniLM embeddings) and a sparse BM25 inverted lexical index.
- **Stage 2: Online Multi-Turn Interactive RAG Application:** Executed via `app.py`. Delivers sub-second response times with zero query-time document re-indexing. Incorporates BCrypt role-based authentication, asynchronous background pre-warming, multi-turn sliding dialogue memory with query reformulation, Hybrid Reciprocal Rank Fusion (RRF, $k=60$), deep neural Cross-Encoder reranking (MS-MARCO), strict anti-hallucination prompting, and automated single-page order PDF extraction.

2. Architectural Workflows & Data Flow Diagrams

The following diagrams illustrate the offline document processing pipeline, online query flow, and file-by-file execution trace.

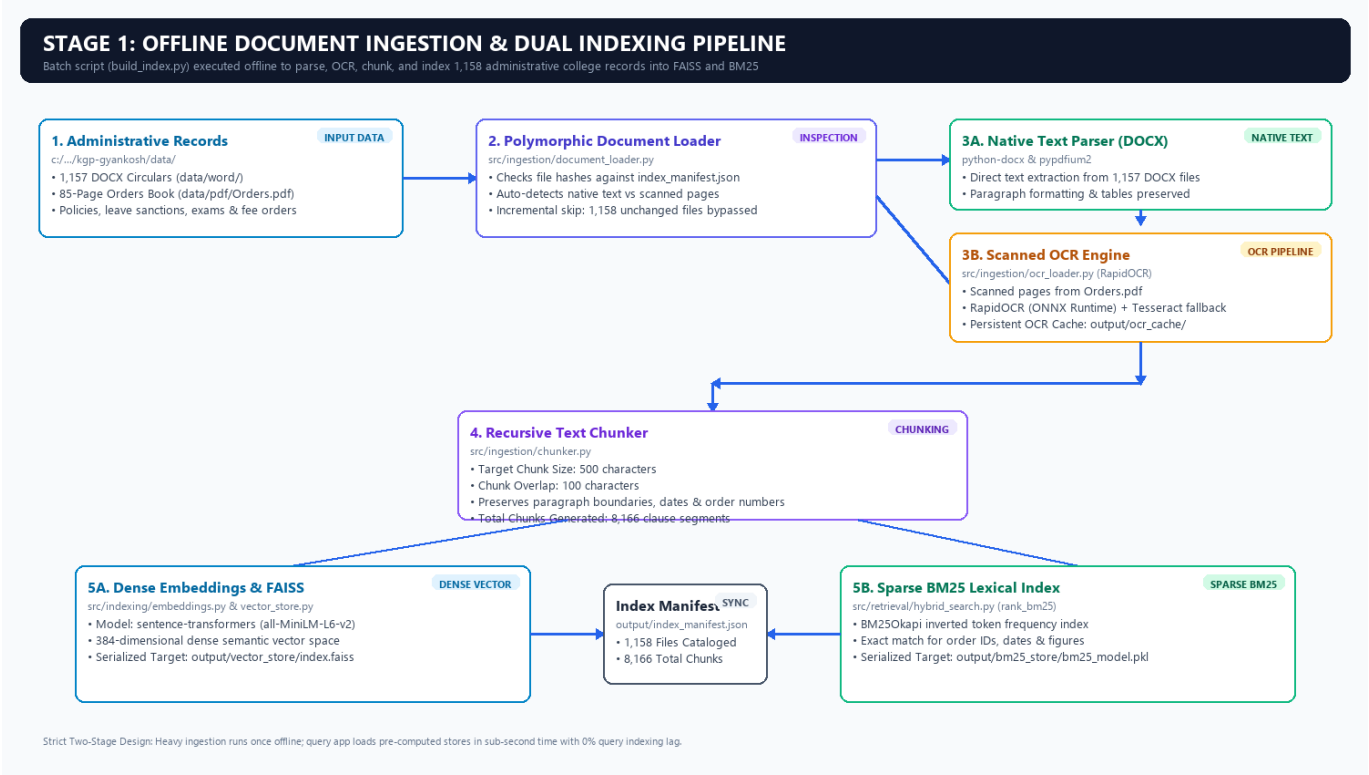
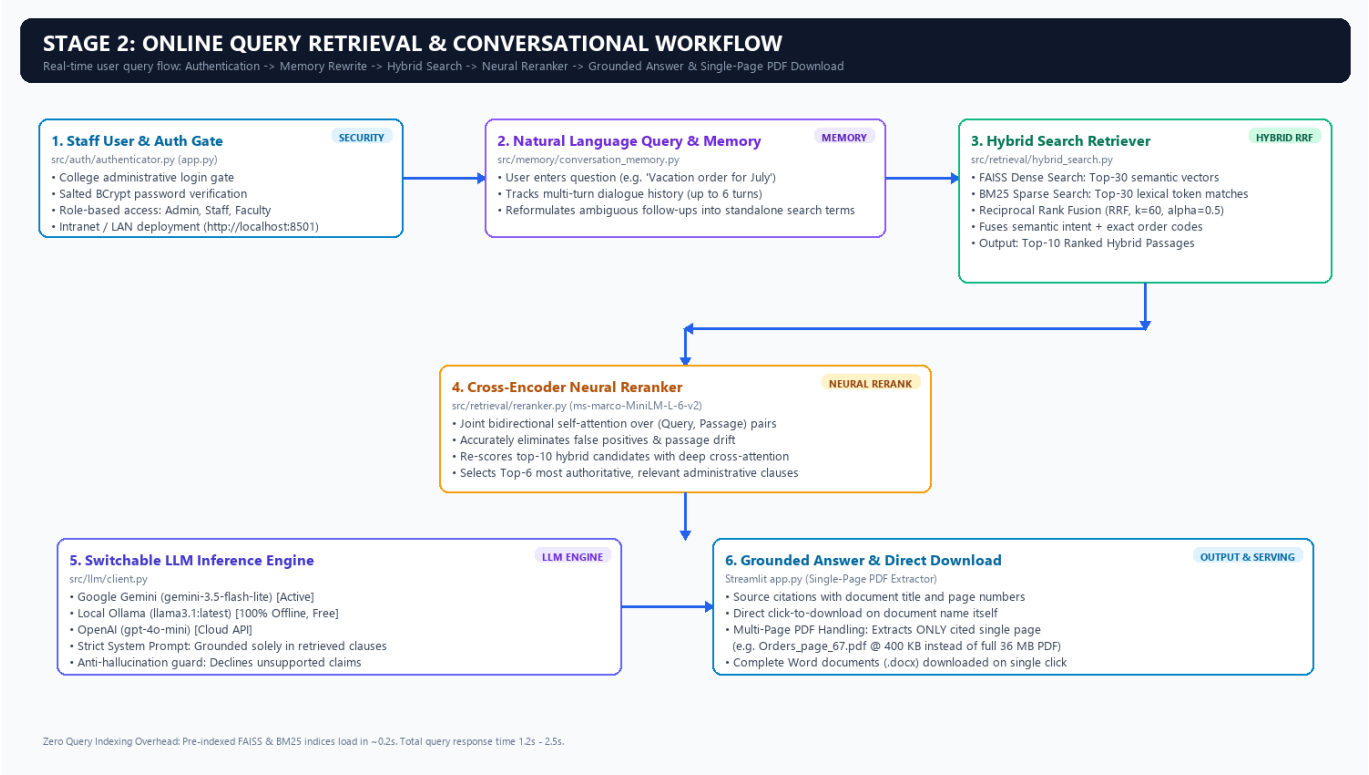
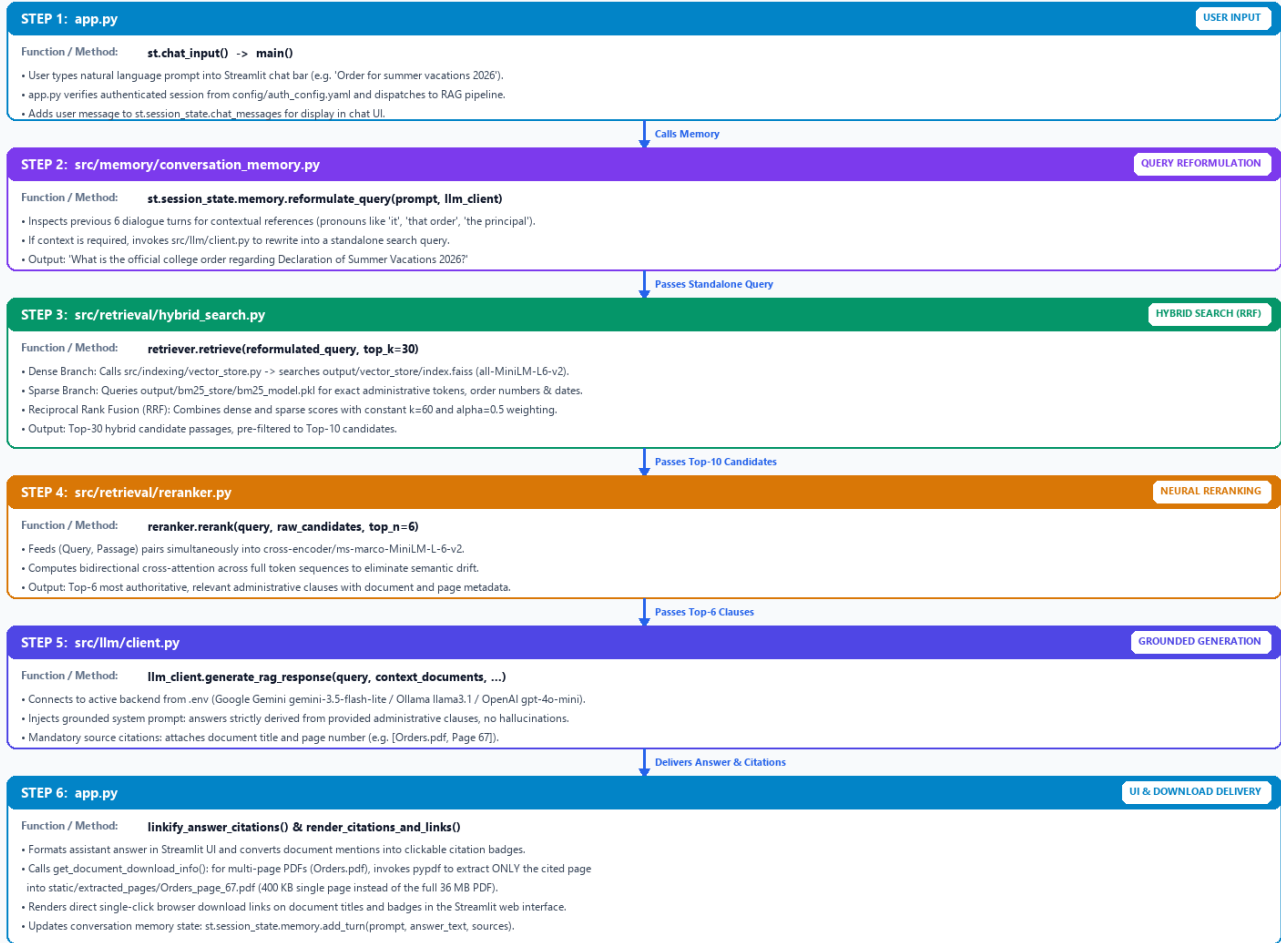


Figure 1: Stage 1 Ingestion, OCR, Chunking, and Dual-Indexing Pipeline (build_index.py)



FILE-BY-FILE CODE EXECUTION FLOW ON USER QUERY

Sequential code execution trace showing exactly which source file, function, and data store executes at each stage



Execution completed in 1.2s - 2.5s. All operations operate on pre-computed local stores with zero query-time re-indexing.

Figure 3: Sequential File-by-File Code Execution Flow (app.py -> memory -> hybrid search -> reranker -> llm)

3. Master File Inventory & Architectural Role

Complete directory of all codebase modules, configuration files, and persistent artifact directories:

File Path	Subsystem	Operational Function & Responsibility
<code>app.py</code>	Frontend / Application	Streamlit web interface, BCrypt login gate, aurora dark glassmorphism theme, background pre-warming, mode selector (RAG vs General AI), auto-scroll, and single-page PDF delivery.
<code>build_index.py</code>	Offline Pipeline	Command-line indexing script. MD5 incremental change detection, polymorphic document extraction, semantic chunking, and FAISS + BM25 disk persistence.
<code>src/auth/authenticator.py</code>	Authentication	Role-based access control (RBAC), salted BCrypt password hashing, and cached YAML credential store loader.
<code>src/ingestion/document_loader.py</code>	Ingestion Engine	Polymorphic document loader supporting DOCX, PDF, TXT, and scanned image formats. Integrates OCR routing.
<code>src/ingestion/ocr_loader.py</code>	OCR Engine	Pure-Python RapidOCR (ONNX Runtime) with optional Tesseract fallback. Disk-based page-level JSON cache.
<code>src/ingestion/chunker.py</code>	Chunking Engine	Recursive character text chunker splitting documents into 500-char blocks with 100-char overlap, preserving administrative clause boundaries.
<code>src/indexing/embeddings.py</code>	Embedding Layer	LangChain embeddings abstraction supporting local SentenceTransformers (all-MiniLM-L6-v2) and OpenAI (text-embedding-3-small).
<code>src/indexing/vector_store.py</code>	Storage Layer	Manages FAISS dense vector store, BM25Okapi lexical index, and MD5 catalog manifest serialization and deserialization.
<code>src/retrieval/hybrid_search.py</code>	Hybrid Retrieval	Combines FAISS dense vector search and BM25 sparse keyword matching using Reciprocal Rank Fusion (RRF, k=60, alpha=0.5).
<code>src/retrieval/reranker.py</code>	Neural Reranking	Deep cross-encoder model (cross-encoder/ms-marco-MiniLM-L-6-v2) performing full self-attention reranking on query-passage pairs.
<code>src/memory/conversation_memory.py</code>	Conversational Memory	Maintains sliding window of last 6 turns and uses LLM for multi-turn contextual query reformulation.
<code>src/llm/client.py</code>	LLM Interface	Multi-backend client (Google Gemini Cloud default, local Ollama, OpenAI) with automatic cloud failover and strict anti-hallucination prompting.
<code>.env</code> & <code>.env.example</code>	Configuration	Environment parameters: active LLM provider, API keys, models, chunk sizes, reranking thresholds, and paths.
<code>.streamlit/config.toml</code>	UI Config	Enables static document serving, headless mode, custom dark theme colors, and browser settings.
<code>config/auth_config.yaml</code>	Credential Store	User credential store containing BCrypt-hashed passwords, emails, full names, and roles (admin, staff, faculty).
<code>requirements.txt</code>	Dependencies	Pinned library dependencies: streamlit, langchain, faiss-cpu, rank-bm25, sentence-transformers, google-genai, rapidocr, reportlab, etc.
<code>data/ (word, pdf)</code>	Data Stores	Raw document repository: 1,157 official polytechnic DOCX circulars + 85-page scanned Orders.pdf archive.
<code>output/ (vector, bm25, ocr)</code>	Index Stores	Serialized indexing artifacts: index.faiss, index.pkl, bm25_model.pkl, bm25_corpus.pkl, index_manifest.json, and OCR cache.

4. Exhaustive File-by-File & Line-by-Line Code Breakdown

The following sections provide an exhaustive, line-by-line and block-by-block technical analysis of every Python module and configuration file comprising the system.

4.1. app.py — Streamlit Frontend & Enterprise Application Engine

app.py is the primary web application entry point (1,244 lines). It orchestrates user authentication, custom glassmorphic styling, background model warming, conversational state management, and real-time query execution.

Lines 1-40: Initialization & Page Configuration

Configures page title, institutional icon (■■■), wide layout, and expanded sidebar. Ensures project root is prepended to sys.path and loads .env.

```
st.set_page_config(
    page_title="KGP Gyankosh - Administrative Intelligence Assistant",
    page_icon="■■■",
    layout="wide",
    initial_sidebar_state="expanded"
)
load_dotenv(os.path.join(PROJECT_ROOT, ".env"))
```

Lines 50-95: Campus Background & Radiant Glassmorphism Atmosphere

Loads static/campus_bg.jpg as base64 and layers radiant cyber-cyan (#38bdf8) and royal violet (#818cf8) radial gradients over a deep obsidian base (#070b14) to create a high-contrast enterprise theme.

```
def load_campus_bg_base64() -> str:
    bg_path = os.path.join(PROJECT_ROOT, "static", "campus_bg.jpg")
    with open(bg_path, "rb") as f:
        return base64.b64encode(f.read()).decode("utf-8")
```

Lines 96-550: Scoped CSS & Non-Blocking Selectbox Dropdown Engine

Scoped text inputs to avoid breaking Streamlit's internal BaseWeb selectbox dropdown. Applies frosted glass styling to dropdown menus with high z-index (99999999), custom radio toggle pills, and vibrant citation badges.

```
div[data-baseweb="select"] > div {
    background: linear-gradient(145deg, rgba(14,22,44,0.95), rgba(18,28,56,0.9)) !important;
    border: 1px solid rgba(56, 189, 248, 0.4) !important;
}
div[data-baseweb="popover"] { z-index: 99999999 !important; }
```

Lines 565-610: Dynamic Single-Page PDF Order Extraction (pypdf)

When an order is cited (e.g. Orders.pdf, Page 67), intercepts the request and extracts ONLY that single page into static/extracted_pages/Orders_page_67.pdf using pypdf. The user downloads a lightweight ~400 KB PDF instead of the full 36 MB orders catalog.

```
from pypdf import PdfReader, PdfWriter
reader = PdfReader(file_path)
writer = PdfWriter()
writer.add_page(reader.pages[page_num - 1])
with open(page_filepath, "wb") as f_out:
    writer.write(f_out)
```

Lines 710-755: Cached System Resource Initialization

Decorated with @st.cache_resource. Loads FAISS index, BM25 store, manifest catalog, reranker, and default Gemini client once into memory, guaranteeing 0 query-time indexing overhead.

```
@st.cache_resource(show_spinner="■ Initializing AI Knowledge Base...")
def initialize_system():
    embeddings = get_embeddings_model()
    vector_store = load_vector_store(vector_store_dir, embeddings)
    bm25_model, bm25_corpus = load_bm25_store(bm25_store_dir)
    return {"retriever": retriever, "reranker": reranker, ...}
```

Lines 820-840: Background Thread Model Pre-Warming

During initial landing page view (while entering credentials), spawns a daemon thread running `initialize_system()`. By the time credentials are submitted, heavy models are already warm in memory, making login instant.

```
if "prewarm_started" not in st.session_state:
    st.session_state.prewarm_started = True
    import threading
    threading.Thread(target=initialize_system, daemon=True).start()
```

Lines 860-880: Memoized LLM Client Factory (_LLM_CLIENTS)

Caches initialized LLM instances by (provider, model_name) key. Eliminates module reloads and allows instant, 0 ms model switching in the dropdown.

```
_LLM_CLIENTS: Dict[str, LLMClient] = {}
def get_cached_llm_client(provider: str, model_name: str) -> LLMClient:
    key = f"{provider}:{model_name}"
    if key in _LLM_CLIENTS:
        return _LLM_CLIENTS[key]
    client = LLMClient(provider=provider, model=model_name)
    _LLM_CLIENTS[key] = client
    return client
```

Lines 1150-1240: Dual-Mode Execution, Auto-Scroll, & Citation Rendering

Toggles between **College Records (RAG)** and **General AI Chat**. Injects smooth auto-scroll JS so queries and answers appear in view without manual scrolling. Citations are linkified to direct download badges.

```
if prompt := st.chat_input(input_placeholder):
    st.session_state.chat_messages.append({"role": "user", "content": prompt})
    scroll_to_bottom()
    candidates = retriever.retrieve(reformulated_query)
    reranked = reranker.rerank(reformulated_query, candidates)
    result = llm_client.generate_answer(prompt, top_documents)
```

4.2. build_index.py — Stage 1 Offline Batch Ingestion & Indexing Engine

build_index.py is the offline batch processing engine (265 lines). It reads documents from data/, computes MD5 file hashes for incremental detection, extracts text, chunks semantic segments, and creates the FAISS vector database and BM25 lexical index.

Lines 40-75: Incremental MD5 Change Detection

Calculates hashlib.md5 hashes for every file in data/. Compares against index_manifest.json. Files with matching hashes are skipped, enabling incremental updates in under 2 seconds.

```
def compute_file_md5(file_path: str) -> str:
    hasher = hashlib.md5()
    with open(file_path, "rb") as f:
        for chunk in iter(lambda: f.read(65536), b''):
            hasher.update(chunk)
    return hasher.hexdigest()
```

Lines 80-160: Batch Processing & Document Chunking Coordinator

Discovers .docx, .pdf, .txt files, invokes DocumentLoader for parsing, passes extracted documents to Chunker (500 chars, 100 overlap), and aggregates chunks.

```
loader = DocumentLoader()
chunker = DocumentChunker(chunk_size=500, chunk_overlap=100)
all_chunks = []
for file_path in discovered_files:
    docs = loader.load(file_path)
    chunks = chunker.split_documents(docs)
    all_chunks.extend(chunks)
```

Lines 165-240: Dual-Index Serialization (FAISS + BM25)

Computes dense embeddings via SentenceTransformers and builds FAISS index. Tokenizes corpus and serializes BM25Okapi model to disk along with manifest catalog.

```
embeddings = get_embeddings_model()
vector_store = FAISS.from_documents(all_chunks, embeddings)
save_vector_store(vector_store, vector_store_dir)
save_bm25_store(all_chunks, bm25_store_dir)
save_manifest(manifest_path, manifest_data)
```

4.3. src/auth/authenticator.py — Access Control & BCrypt Security Gate

src/auth/authenticator.py enforces internal role-based access control (RBAC). Uses bcrypt-hashed passwords and manages encrypted cookie sessions via streamlit-authenticator.

Lines 18-35: Password Hashing & Cached Config Loader

Provides bcrypt hashing helper. Uses @st.cache_data to prevent repetitive disk I/O on every Streamlit rerun.

```
def hash_password(password: str) -> str:
    return bcrypt.hashpw(password.encode("utf-8"), bcrypt.gensalt()).decode("utf-8")

@st.cache_data(show_spinner=False)
def load_auth_config(config_path=None) -> dict:
    with open(path, "r", encoding="utf-8") as f:
        return yaml.safe_load(f)
```

4.4. Ingestion Subsystem: document_loader.py, ocr_loader.py, chunker.py

src/ingestion/document_loader.py: Polymorphic Ingestion

Detects file format. Extracts text and tables from Word (.docx) using python-docx. Extracts native PDF text using pypdf. Routes scanned / image pages to OCRLoader.

```
if ext == ".docx": return self._load_docx(file_path)
elif ext == ".pdf": return self._load_pdf(file_path)
elif ext in [".png", ".jpg", ".jpeg"]: return self.ocr_loader.load(file_path)
```

src/ingestion/ocr_loader.py: RapidOCR Engine & Persistent Page Caching

Uses Pure-Python RapidOCR (ONNX Runtime). Maintains persistent JSON cache in output/ocr_cache/ so the 85-page Orders.pdf is OCR-processed only once across rebuilds.

```
cache_key = f"{os.path.basename(file_path)}_pages.json"
if os.path.exists(cache_file):
    return self._load_cache(cache_file)
# Extract and run RapidOCR per page
ocr = RapidOCR()
result, _ = ocr(page_image)
```

src/ingestion/chunker.py: Recursive Character Splitting

Uses LangChain RecursiveCharacterTextSplitter with separators ["\n\n", "\n", ". ", " "]. Guarantees that administrative clauses, fee tables, and order sections remain intact.

```
splitter = RecursiveCharacterTextSplitter(
    chunk_size=500, chunk_overlap=100,
    separators=["\n\n", "\n", ". ", " "]
)
```

4.5. Indexing Subsystem: embeddings.py, vector_store.py

src/indexing/embeddings.py: Unified Embeddings Provider

Switchable via EMBEDDING_PROVIDER (.env). Default: local sentence-transformers (all-MiniLM-L6-v2) running on CPU with normalized vectors for cosine distance.

```
class DirectSentenceTransformersEmbeddings(Embeddings):
    def embed_documents(self, texts):
        return self.model.encode(texts, normalize_embeddings=True).tolist()
    def embed_query(self, text):
        return self.model.encode([text], normalize_embeddings=True)[0].tolist()
```

src/indexing/vector_store.py: FAISS Vector Store & BM25 Storage

Manages index.faiss persistence and BM25Okapi tokenized index serialization via pickle.

```
vector_store = FAISS.load_local(store_dir, embeddings, allow_dangerous_deserialization=True)
with open(model_path, "wb") as f: pickle.dump(bm25_model, f)
```

4.6. Retrieval Subsystem: `hybrid_search.py`, `reranker.py`

`src/retrieval/hybrid_search.py`: Reciprocal Rank Fusion (RRF)

Retrieves top-k from FAISS and top-k from BM25. Combines ranks using RRF formula: $\text{Score}(d) = \sum (1 / (k + \text{rank}_i))$. Ensures exact order numbers and semantic concepts both score highly.

```
def rrf(dense_results, sparse_results, k=60, alpha=0.5):
    rrf_scores = defaultdict(float)
    for rank, (doc, _) in enumerate(dense_results):
        rrf_scores[doc.metadata['chunk_id']] += alpha * (1.0 / (k + rank + 1))
    for rank, (doc, _) in enumerate(sparse_results):
        rrf_scores[doc.metadata['chunk_id']] += (1 - alpha) * (1.0 / (k + rank + 1))
    return sorted(candidates, key=lambda x: rrf_scores[x.metadata['chunk_id']], reverse=True)
```

`src/retrieval/reranker.py`: Deep Neural Cross-Encoder Reranking

Uses cross-encoder/ms-marco-MiniLM-L-6-v2. Jointly evaluates [query, passage] through bidirectional transformer attention layers, re-scoring top-10 candidates down to top-6 authoritative passages.

```
pairs = [[query, doc.page_content] for doc, _ in candidates]
scores = model.predict(pairs)
for (doc, _), score in zip(candidates, scores):
    doc.metadata["rerank_score"] = float(score)
    reranked.append((doc, float(score)))
return reranked[:limit]
```

4.7. Intelligence Subsystem: `conversation_memory.py`, `client.py`

`src/memory/conversation_memory.py`: Sliding Window Buffer & Query Reformulation

Stores last 6 conversation turns. When follow-up questions are submitted (e.g. 'When does it expire?'), uses the LLM to resolve pronoun references into a standalone query for retrieval.

```
prompt = REFORMULATION_PROMPT.format(history=recent_history, question=query)
standalone_query = llm_client.generate_raw(prompt).strip()
```

`src/llm/client.py`: Switchable Backend, Anti-Hallucination & Auto-Fallback

Enforces strict administrative grounding prompt. If context does not answer the query, replies: 'I could not find this information in the available official documents.' Includes automatic fallback to Google Gemini Cloud if local Ollama runs out of RAM.

```
ADMIN_SYSTEM_PROMPT = """You are KGP Gyankosh... Ground answers STRICTLY in context. If unmentioned, state 'I could not find this information in official documents.' Conclude with Sources: - [Doc, Page X]"""

# Auto-fallback to Gemini if local Ollama fails (out of RAM)
if self.provider == 'ollama' and google_key:
    fallback_llm = ChatGoogleGenerativeAI(model=gemini_model, ...)
```

5. Deployment & Execution Commands

1. Run the interactive Streamlit assistant:

```
streamlit run app.py
```

2. Rebuild offline document index from data/:

```
python build_index.py
```

3. Run automated retrieval test suite:

```
pytest tests/
```

4. Re-compile this technical documentation PDF:

```
python scripts/generate_line_by_line_doc_pdf.py
```